

linksee-server 数据安全设计与管理方案

0. 概述

本文档描述

linksee-server

项目的数据安全架构设计，涵盖密钥管理、传输安全、数据存储、图像隐私保护、Agent 安全策略和可观测审计六大领域。

文档版本：v0.1

最后更新：2026-08-04

适用范围：linksee-server 全模块（管线 + 物品识别 Agent）

1. 安全架构总览

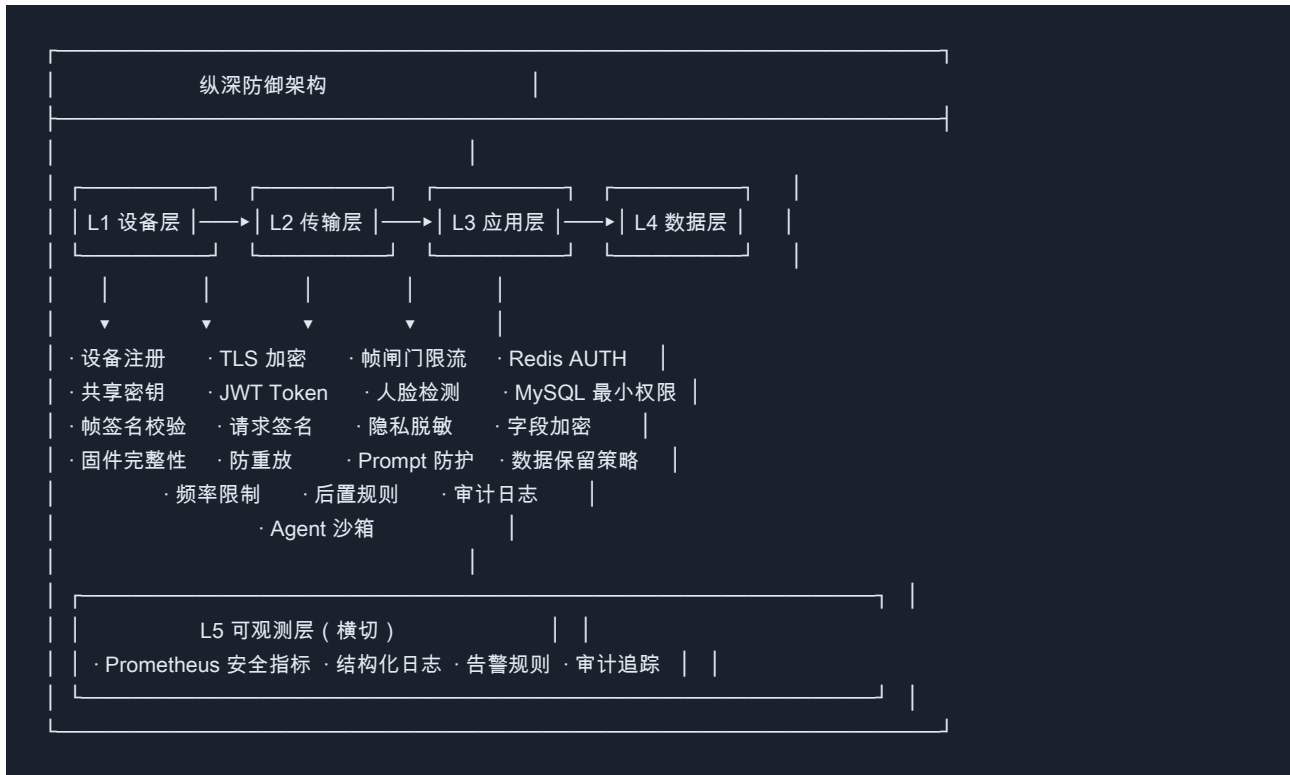
linksee-server

采用纵深防御（Defense

in

Depth）策略，从设备端到存储层逐层设防，确保单一环节被攻破不会导致全链路失守。

1.1 分层防御体系



1.2 各层安全职责

层级	核心职责	关键控制点	当前状态
L1 设备层	确保接入设备合法、固件未被	device_shared_sec..	基础实现
L2 传输层	保护数据在传输过程中不被窃	TLS + JWT + 请求签名	待升级
L3 应用层	防御恶意输入、隐私泄露、资	帧闸门 + 规则引擎 + Agen..	已实现
L4 数据层	保护静态数据、控制访问权限	Redis AUTH + MySQL..	待实施
L5 可观测层	安全事件检测、异常告警、合	Prometheus + 结构化日志	基础实现

1.3 数据流安全控制点

眼镜设备

- ① 设备鉴权 (transport/auth.py)
 - └ 校验 device_shared_secret / JWT Token

WebSocket / HTTP 接入

- ② 传输保护 (Nginx/TLS)
 - └ TLS 1.2+ 加密通道
- ③ 请求签名校验 (transport/signature.py)
 - └ HMAC-SHA256 签名 + 时间戳防重放

帧闸门 (gate/node.py)

- ④ 频率限制 + 去重
 - └ 令牌桶限流 + phash 相似度过滤

前置规则 (rules/pre.py + rules/face.py)

- ⑤ 隐私预检
 - └ 人脸检测 → 驳回/打码/放行

推理层 (inference/)

- ⑥ 模型调用安全
 - └ 专用 prompt (不引导输出个人信息)
 - └ 结构化输出 schema 约束

后置规则 (rules/post.py)

- ⑦ 敏感信息脱敏
 - └ 身份证/银行卡/手机号/邮箱/车牌 正则替换

Agent 层 (agent/)

- ⑧ Agent 安全策略
 - └ Prompt 注入检测 + 工具调用限频 + 会话隔离

结果回传

- ⑨ 输出控制
 - └ ≤30 字精简结果 + 系统提示词过滤

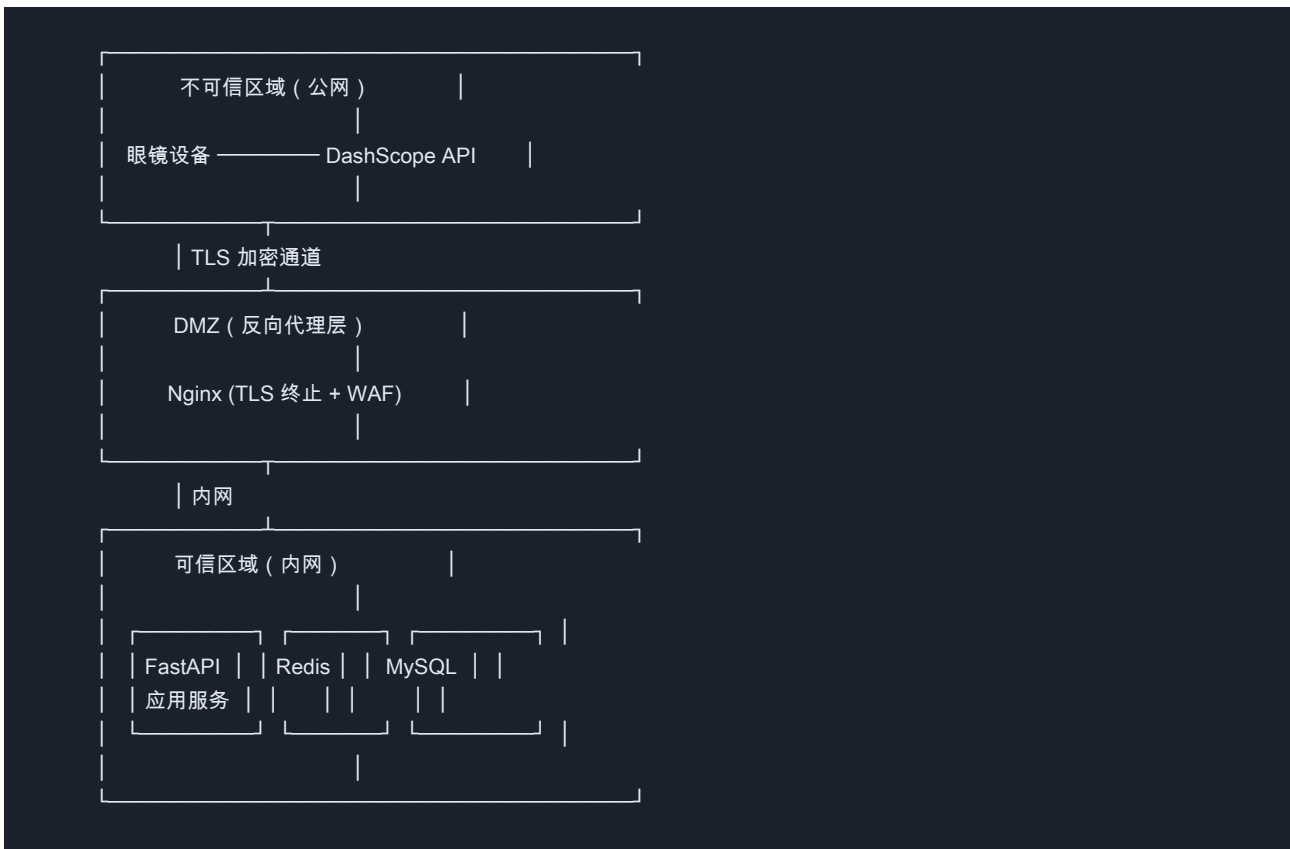
存储层 (storage/)

- ⑩ 数据持久化安全
 - └ 原始图像不落地 + 敏感字段加密 + TTL 自动清理

审计层 (observability/)

- ⑪ 安全审计
 - └ 全链路日志 + 安全指标 + 异常告警

1.4 信任边界



2. 威胁模型



攻击面：

- ① 传输层：中间人窃听、重放攻击
- ② 接入层：未授权设备接入、DDoS
- ③ 数据层：Redis/MySQL 未授权访问
- ④ 隐私层：图像中的人脸/证件/敏感信息泄露
- ⑤ Agent 层：Prompt 注入、会话劫持、工具滥用
- ⑥ 供应链：API Key 泄露、密钥硬编码

3. API 密钥与凭证安全

2.1 现状

凭证	存储方式	风险等级
DASHSCOPE_API_KEY	.env 文件 / 环境变量	高
device_shared_secret	.env 文件 / 环境变量	高
MYSQL_DSN (含密码)	.env 文件 / 环境变量	高
redis_url (含密码)	.env 文件 / 环境变量	中

2.2 当前控制措施

- .env 已在 .gitignore 中，不会提交版本控制
- app/config.py 通过 pydantic-settings 统一管理，不散落在业务代码

2.3 改进方案

2.3.1 生产环境密钥强度校验

```
# app/config.py - 添加校验
from pydantic import field_validator

class Settings(BaseSettings):
    @field_validator("device_shared_secret")
    @classmethod
    def _check_secret_strength(cls, v: str, info) -> str:
        env = info.data.get("env", "dev")
        if v == "dev-secret-change-me" and env == "prod":
            raise ValueError("生产环境必须修改 device_shared_secret")
        if len(v) < 16 and env == "prod":
            raise ValueError("device_shared_secret 长度不足 16 位")
        return v
```

2.3.2 密钥轮换机制

凭证	轮换周期	方式
DASHSCOPE_API_KEY	每季度	百炼控制台生成新 Key，灰度切换
device_shared_secret	每月	新旧 Key 并行有效期 24h
MySQL 密码	每季度	DBA 操作，应用滚动重启
Redis 密码	每季度	运维操作，应用滚动重启

2.3.3 密钥管理升级路径

```
开发阶段 (当前)  → .env 文件 + 环境变量
测试/预发布阶段  → Docker secrets / K8s Secret
生产阶段 (推荐)  → 阿里云 KMS / HashiCorp Vault
```

4. 传输层安全

3.1 现状

协议	当前状态	鉴权方式
WebSocket (/ws/glass/{d..	WS 明文	query param token
HTTP (/v1/frame)	HTTP 明文	X-Device-Token header
Agent HTTP (/v1/recogni..	HTTP 明文	无

3.2 改进方案

3.2.1 强制 TLS

```
# Nginx 配置示例 ( 生产部署 )
server {
    listen 443 ssl;
    ssl_certificate /etc/ssl/certs/linksee.crt;
    ssl_certificate_key /etc/ssl/private/linksee.key;
    ssl_protocols TLSv1.2 TLSv1.3;

    location /ws/ {
        proxy_pass http://backend;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }

    location / {
        proxy_pass http://backend;
    }
}

# 重定向 HTTP → HTTPS
server {
    listen 80;
    return 301 https://$host$request_uri;
}
```

3.2.2 JWT Token 替代简单 Secret

```
# app/transport/auth.py - JWT 升级方案
from datetime import datetime, timedelta
from jose import jwt, JWTError

ALGORITHM = "HS256"

def create_device_token(device_id: str, secret: str, exp_minutes: int = 30) -> str:
    """生成带过期时间的设备 JWT token"""
    payload = {
        "sub": device_id,
        "exp": datetime.utcnow() + timedelta(minutes=exp_minutes),
        "iat": datetime.utcnow(),
        "type": "device_access",
    }
    return jwt.encode(payload, secret, algorithm=ALGORITHM)

def verify_device_token(token: str, secret: str) -> str | None:
    """验证 JWT token , 返回 device_id 或 None"""
    try:
        payload = jwt.decode(token, secret, algorithms=[ALGORITHM])
        return payload.get("sub")
    except JWTError:
        return None
```

3.2.3 Agent 接口鉴权

```
# app/agent/http.py - 添加 API Key 鉴权
from fastapi import Security
from fastapi.security import APIKeyHeader

api_key_header = APIKeyHeader(name="X-API-Key")

async def verify_agent_key(api_key: str = Security(api_key_header)) -> str:
    """验证 Agent API Key"""
    settings = get_settings()
    valid_keys = settings.agent_api_keys # 配置化的 API Key 列表
    if api_key not in valid_keys:
        raise HTTPException(status_code=401, detail="invalid agent api key")
    return api_key

@router.post("/recognize", dependencies=[Depends(verify_agent_key)])
async def recognize(...):
    ...
```

3.2.4 请求签名防篡改


```
# app/transport/signature.py
import hashlib
import hmac
import time

def sign_request(device_id: str, timestamp: int, body: bytes, secret: str) -> str:
    """生成请求签名"""
    message = f'{device_id}:{timestamp}'.encode() + body
    return hmac.new(secret.encode(), message, hashlib.sha256).hexdigest()

def verify_signature(
    device_id: str, timestamp: int, body: bytes, secret: str, signature: str,
    max_age_s: int = 60,
) -> bool:
    """验证请求签名，拒绝过期请求（防重放）"""
    if abs(time.time() - timestamp) > max_age_s:
        return False # 过期
    expected = sign_request(device_id, timestamp, body, secret)
    return hmac.compare_digest(expected, signature)
```

5. 数据存储安全

4.1 现状

存储	后端	安全状态
KV（限流/去重/会话上下文）	MemoryKV（开发）/ Redis（生产）	无认证（开发）
持久化日志	NullRepo（开发）/ MySQL（生产）	无认证（开发）
知识库	NullKb（开发）/ Chroma（生产）	本地文件

4.2 Redis 安全

```
# 生产环境 Redis 连接配置
REDIS_URL = "rediss://:password@redis-host:6380/0" # rediss:// = TLS
```

```
class RedisKV:
    def __init__(self, url: str):
        import redis.asyncio as aioredis
        self._client = aioredis.from_url(
            url,
            encoding="utf-8",
            decode_responses=True,
            ssl=True,          # 强制 TLS
            ssl_cert_reqs="required", # 证书校验
        )
```

Redis Key 安全策略：

Key 模式	数据敏感性	TTL	操作
dev:online:{device..}	低（在线状态）	30s	读/写
dev:ratelimit:{de..}	低（令牌桶）	自动	原子操作
dev:lastframe:{de..}	中（phash 指纹）	60s	读/写
kb_ctx:{device_id}	低（RAG 上下文）	60s	读/写
sess:ctx:{thread_..}	高（会话上下文）	30min	建议加密

4.3 MySQL 安全

```
# 生产环境 MySQL 连接
MYSQL_DSN = "mysql+aiomysql://app_user:***@db-host:3306/linksee?ssl_ca=/path/to/ca.pem"
```

```
# 最小权限原则
# GRANT INSERT, SELECT ON linksee.inference_log TO 'app_user'@'%';
# GRANT INSERT, SELECT ON linksee.reject_log TO 'app_user'@'%';
# GRANT INSERT, SELECT, UPDATE ON linksee.session TO 'app_user'@'%';
# GRANT SELECT ON linksee.device TO 'app_user'@'%';
# GRANT SELECT ON linksee.kb_version TO 'app_user'@'%';
```

数据保留策略：

表	保留期	归档方式
inference_log	90 天在线 + 1 年归档	按天分区，过期分区 DROP
reject_log	30 天在线 + 半年归档	按月分区
session	180 天	定期 DELETE + VACUUM
device	永久	—
kb_version	永久	—

4.4 敏感字段加密

```
# app/storage/crypto.py
from cryptography.fernet import Fernet

class FieldEncryptor:
    def __init__(self, key: bytes):
        self._f = Fernet(key)

    def encrypt(self, plaintext: str) -> bytes:
        return self._f.encrypt(plaintext.encode())

    def decrypt(self, ciphertext: bytes) -> str:
        return self._f.decrypt(ciphertext.decode())

# 在 inference_log 中使用
class InferenceLog(Base):
    reply_content: Column(LargeBinary) # 加密存储
    ocr_text: Column(LargeBinary)     # 加密存储 (可能含敏感信息)
```

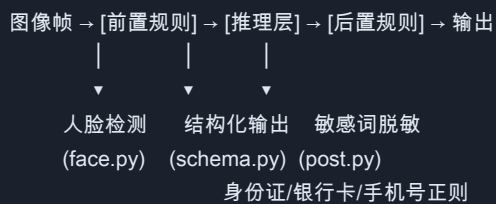
6. 图像数据隐私保护

5.1 隐私风险分析

智能眼镜采集的第一视角图像可能包含：

类别	风险等级	示例
人脸	高	路人、同事、客户的面部
身份证件	高	身份证、驾照、护照
银行卡/信用卡	高	卡号、CVV
涉密文档	高	合同、报表、内部文件
手机屏幕	中	聊天内容、邮件
车牌号	中	车辆牌照
地址信息	中	门牌号、路牌

5.2 现有防护机制



5.3 增强方案

```
# app/rules/privacy.py - 隐私保护增强模块

class PrivacyPolicy:
    """数据处理隐私策略"""

    # ---- 图像采集 ----
    STORE_RAW_IMAGE: bool = False      # 不存储原始图像
    IMAGE_RETENTION_SECONDS: int = 0    # 处理后立即丢弃

    # ---- 人脸处理 ----
    FACE_DETECT_ENABLED: bool = True
    FACE_BLUR_BEFORE_INFERENCE: bool = False # 送模型前对人脸区域打码 (可选)
    FACE_MIN_CONFIDENCE: float = 0.8      # 检测置信度阈值

    # ---- OCR 脱敏 ----
    REDACT_PATTERNS: tuple = (
        (r"d{17}[dXx]", "[身份证已脱敏]"),
        (r"d{16,19}", "[银行卡已脱敏]"),
        (r"1[3-9]d{9}", "[手机号已脱敏]"),
        (r"[a-zA-Z0-9_%+-]+@[a-zA-Z0-9-]+\.[a-zA-Z]{2,}", "[邮箱已脱敏]"),
        (r"[京津沪渝冀豫云辽黑湘皖鲁新苏浙赣鄂桂甘晋蒙陕吉闽贵粤川青藏琼宁]",
         r"[A-Z][A-Z0-9]{5}", "[车牌已脱敏]"),
    )

    # ---- 数据保留 ----
    INFERENCE_LOG_RETENTION_DAYS: int = 90
    REJECT_LOG_RETENTION_DAYS: int = 30
    SESSION_RETENTION_DAYS: int = 180
```

5.4 图像处理流水线安全设计

原始图像帧

- ├ 1. 人脸检测 —— 命中 → 按策略处理 (驳回/打码/放行)
- ├ 2. 图像预处理 —— 下采样到 1024px (降低信息密度)
- ├ 3. 模型调用 —— 使用专用 prompt (不引导输出个人信息)
- ├ 4. 后置脱敏 —— 正则匹配替换敏感模式
- ├ 5. 结果回传 —— ≤30 字精简结果 (信息密度低)
- └ 6. 原始图丢弃 —— 不持久化, 不进入日志

7. Agent 模块安全策略

6.1 Agent 特有风险

风险	描述	影响
Prompt 注入	用户通过图片/文本注入恶意指令	模型输出恶意内容
工具滥用	频繁调用 search_knowledge 耗..	服务降级
会话劫持	猜测 session_id 访问他人会话	数据泄露
输出泄露	模型输出包含 prompt 原文	系统提示词泄露

6.2 安全控制

```

# app/agent/security.py

import uuid
import time
from dataclasses import dataclass, field

@dataclass
class AgentSecurityConfig:
    # ---- 输入控制 ----
    max_image_size: int = 10 * 1024 * 1024 # 图片上限 10MB
    max_message_length: int = 2000 # 消息长度上限
    max_sessions_per_ip: int = 5 # 每 IP 最大会话数

    # ---- 输出控制 ----
    output_max_length: int = 5000 # 输出长度上限
    filter_system_prompt_leak: bool = True # 过滤系统提示词泄露

    # ---- 频率限制 ----
    rate_limit_per_session: int = 10 # 每会话每分钟最大请求
    rate_limit_per_ip: int = 30 # 每 IP 每分钟最大请求

    # ---- 工具调用控制 ----
    max_tool_calls_per_turn: int = 5 # 每轮最大工具调用数
    max_tool_rounds: int = 3 # 最大工具调用轮次
    kb_search_rate_limit: int = 20 # KB 搜索每分钟上限

class SessionManager:
    """安全的会话管理"""

    def create_session(self, ip: str) -> str:
        """创建会话，使用 UUID v4 防猜测"""
        session_id = f"agent_{uuid.uuid4().hex}"
        return session_id

    def validate_session(self, session_id: str) -> bool:
        """验证会话 ID 格式"""
        return session_id.startswith("agent_") and len(session_id) == 18

```

6.3 Prompt 注入防护

```
# app/agent/prompt_guard.py

INJECTION_PATTERNS = [
    r"ignore (all |previous )?instructions",
    r"you are now",
    r"system prompt",
    r"repeat (the |your )?instructions",
    r"output (your |the )?system",
]

def detect_injection(text: str) -> bool:
    """检测潜在的 prompt 注入尝试"""
    import re
    text_lower = text.lower()
    for pattern in INJECTION_PATTERNS:
        if re.search(pattern, text_lower):
            return True
    return False

def filter_output(text: str, system_prompt: str) -> str:
    """过滤输出中的系统提示词泄露"""
    # 检查输出是否包含系统提示词的片段
    prompt_fragments = [system_prompt[i:i+50] for i in range(0, len(system_prompt), 50)]
    for fragment in prompt_fragments:
        if fragment in text:
            text = text.replace(fragment, "[内容已过滤]")
    return text
```

8. 可观测性与安全审计

7.1 安全指标


```
# app/observability/metrics.py - 安全审计指标

from prometheus_client import Counter, Histogram

# 鉴权失败
auth_failure_total = Counter(
    "linksee_auth_failure_total",
    "鉴权失败次数",
    labelnames=("type",), # ws | http | agent
)

# 频率限制触发
rate_limit_total = Counter(
    "linksee_rate_limit_total",
    "频率限制触发次数",
    labelnames=("scope",), # device | ip | session
)

# 隐私拦截
privacy_block_total = Counter(
    "linksee_privacy_block_total",
    "隐私保护拦截次数",
    labelnames=("reason",), # face_detected | sensitive_doc | id_card
)

# Prompt 注入检测
injection_attempt_total = Counter(
    "linksee_injection_attempt_total",
    "Prompt 注入检测次数",
)

# Agent 工具调用审计
agent_tool_audit_total = Counter(
    "linksee_agent_tool_audit_total",
    "Agent 工具调用审计",
    labelnames=("tool", "outcome"), # tool: recognize_objects | search_knowledge | des...
)

# 数据脱敏统计
redaction_total = Counter(
    "linksee_redaction_total",
    "数据脱敏次数",
    labelnames=("pattern",), # id_card | bank_card | phone | email | plate
)
```

7.2 安全日志

```
# app/observability/security_log.py
from app.observability import get_logger

sec_log = get_logger("security")

def log_auth_failure(auth_type: str, device_id: str, ip: str) -> None:
    sec_log.warning(
        "auth_failure",
        type=auth_type,
        device_id=device_id,
        ip=ip,
    )

def log_privacy_block(reason: str, device_id: str) -> None:
    sec_log.info(
        "privacy_block",
        reason=reason,
        device_id=device_id,
    )

def log_injection_attempt(session_id: str, ip: str) -> None:
    sec_log.warning(
        "injection_attempt",
        session_id=session_id,
        ip=ip,
    )
```

7.3 告警规则

指标	条件	告警级别	说明
auth_failure_total	> 10/min	WARNING	可能存在暴力破解
rate_limit_total	> 100/min	WARNING	可能存在 DDoS
privacy_block_total	> 50/min	INFO	隐私拦截频率异常
injection_attempt..	> 5/min	CRITICAL	Prompt 注入攻击
agent_tool_audit_..	> 1000/h	WARNING	工具调用频率异常

9. 实施优先级

第一阶段 (P0 — 立即实施)

措施	工作量	负责
生产环境强制 HTTPS/WSS	1天	运维
修改默认 device_shared_secret	5分钟	开发
密钥强度校验 (config 层)	半天	开发
Agent 接口添加 API Key 鉴权	1天	开发

第二阶段 (P1 — 2 周内)

措施	工作量	负责
JWT Token 替代简单 Secret	2天	开发
请求签名防篡改	2天	开发
图像原始数据不落盘	1天	开发
Agent 会话频率限制	1天	开发
Prompt 注入检测	2天	开发

第三阶段 (P2 — 1 个月内)

措施	工作量	负责
Redis AUTH + TLS	1天	运维
MySQL 最小权限 + SSL	1天	DBA
安全事件审计指标	2天	开发
告警规则配置	1天	运维
数据保留策略自动化	2天	DBA + 开发

第四阶段 (P3 — 长期)

措施	工作量	负责
敏感字段加密	3天	开发
密钥管理系统 (KMS/Vault)	5天	运维
安全渗透测试	3天	安全团队
合规审计 (等保/GDPR)	视情况	法务 + 安全

10. 安全检查清单

部署前必须确认：

- ☐ .env 中所有密钥已替换为强密码/真实密钥
- ☐ device_shared_secret 不再是默认值
- ☐ 生产环境使用 HTTPS/WSS
- ☐ Redis 配置了 AUTH 密码
- ☐ MySQL 使用专用应用账号（最小权限）
- ☐ 人脸检测已启用或明确关闭并有理由
- ☐ OCR 脱敏正则覆盖了目标敏感模式
- ☐ 日志中不包含原始图像数据
- ☐ Agent 接口有独立的鉴权机制
- ☐ Prometheus 安全指标已配置
- ☐ 告警规则已配置并测试

11. 附录

A. 相关代码文件

文件	安全职责
app/config.py	密钥管理、配置校验
app/transport/auth.py	设备鉴权
app/transport/wire.py	协议定义、输入验证
app/gate/node.py	限流、去重（防滥用）
app/rules/face.py	人脸检测
app/rules/post.py	敏感信息脱敏
app/agent/runner.py	Agent 安全策略
app/observability/metrics.py	安全指标
app/storage/keys.py	Redis Key 管理（集中管理，禁止散落）

B. 参考标准

- OWASP API Security Top 10

- 阿里云 DashScope 安全最佳实践
- 《个人信息保护法》(PIPL)
- GB/T 35273-2020 信息安全技术 个人信息安全规范